

Conversation Summarization Specification for *Ragtime-Pro – Multi-Turn Memory via R2*

1. Purpose

Conversation summarization ensures that multi-turn interactions remain:

- **contextually coherent**
- **token-efficient**
- **scalable**
- **serverless-friendly**

The summarization subsystem compresses older conversation turns into a concise semantic summary stored in R2. This allows the system to maintain long-term memory without exceeding token limits or degrading retrieval quality.

2. When Summarization Occurs

Summarization is triggered when **any** of the following conditions are met:

2.1 History Length Threshold

If `history.length > MAX_HISTORY_LENGTH` Recommended default: **20 messages**

2.2 Token Count Threshold

If combined history tokens exceed **1500–2000 tokens**

2.3 Explicit User Request

If user says:

- “Summarize our conversation”
- “Give me a recap”
- “What have we discussed so far”

2.4 Session Restoration

When a user returns (via cookie or fingerprint match), the system may summarize the previous session before continuing.

3. Summarization Strategy

Summarization uses **Azure OpenAI** to generate a compressed representation of the conversation.

3.1 Summary Goals

The summary must:

- Capture **intent**, **goals**, and **context**
- Preserve **facts** the user has provided
- Preserve **assistant commitments**
- Exclude irrelevant chit-chat
- Be **stable** across turns (not drifting)
- Be **short** (target: 150–250 tokens)

3.2 Summary Format

Stored in R2 as a single string:

json

"summary": "User is exploring Ragtime meta-agent architecture, resolver design, and hybrid retrieval. They want deterministic workflows and structured reasoning. Assistant has explained chunking, embeddings, hybrid scoring, and session management."

4. Summarization Prompt (Claude must implement)

System Prompt

You are a conversation summarization model. Your task is to compress the conversation into a concise, factual summary that preserves the user's goals, context, and constraints. Do not include irrelevant small talk. Do not invent details. Capture only what is necessary for future turns.

User Prompt Template

Summarize the following conversation so it can be used as context in future turns.

Conversation:

{{history}}

Requirements:

- Preserve user goals and constraints.
- Preserve assistant commitments.
- Preserve important facts.
- Remove irrelevant details.

- Keep the summary under 250 tokens.
- Output only the summary text.

Claude should generate the summary and return it as plain text.

5. Summarization Workflow

5.1 Load Session

```
session = readSession(sessionId)
```

5.2 Check Thresholds

If `history.length > 20` or `token count > 1500`:

→ Trigger summarization.

5.3 Build Summarization Prompt

Insert:

- `session.summary` (if exists)
- `session.history` (full list)

5.4 Call Azure OpenAI

Claude must implement the call.

5.5 Update Session File

After receiving summary:

```
session.summary = <newSummary>
```

```
session.history = last N messages (e.g., last 4)
```

```
session.lastSeen = now
```

```
writeSession(session)
```

This keeps the session lightweight.

6. How Summary Is Used in Retrieval

When generating the final answer:

6.1 Include Summary in Prompt

Claude must prepend:

Conversation summary:

```
{{session.summary}}
```

6.2 Include Recent Turns

Include last few messages from session.history.

6.3 Do NOT feed summary into dense/sparse retrieval

Summary is only for LLM reasoning, not for RAG retrieval.

7. Summary Stability Requirements

Claude must ensure:

7.1 No Drift

Summary must not change meaning across turns.

7.2 No Overwriting of User Intent

If user changes goals, summary must reflect the new goals.

7.3 No Loss of Critical Information

Critical facts must remain in summary until user explicitly invalidates them.

7.4 No Redundant Growth

Summary must remain compact (≤ 250 tokens).

8. Error Handling

8.1 Summarization Failure

If Azure OpenAI fails:

- Keep existing summary
- Truncate history to last N messages
- Log error (Claude must implement logging)

8.2 Corrupted Summary

If summary is empty or malformed:

- Regenerate summary from full history

8.3 Missing Summary

If summary == "":

- Generate summary immediately

9. R2 Storage Structure

Stored under:

conversations/{sessionId}.json

Example:

json

```
{
  "sessionId": "uuid",
  "fingerprint": "hashed-ip-ua",
  "summary": "User is exploring Ragtime meta-agent architecture...",
  "history": [
    { "role": "user", "content": "Explain Ragtime architecture." },
    { "role": "assistant", "content": "Ragtime is a meta-agent framework..." }
  ],
  "lastSeen": "2026-08-13T15:18:00Z"
}
```

10. Deliverables for Claude

Claude must implement:

1. **Summarization trigger logic**
2. **Summarization prompt builder**
3. **Azure OpenAI summarization call**
4. **Summary storage in R2**
5. **History truncation logic**
6. **Summary injection into final answer prompts**
7. **Error handling and fallback behavior**